🔐 Login com JWT no Next.js

Estrutura completa de autenticação no Next.js 15/16

Continuação direta da aula `06.JWT` **(Backend)** — agora implementando o **lado do cliente**

App Router | Server Actions | Cookie httpOnly | Middleware

🔗 Conexão com a aula de JWT (Backend)

Na aula anterior vimos que o **JWT** é um crachá digital assinado:

`eyJhbGciOi...` . `eyJzdWIiOi...` . `SflKxwRJ...`

1. Header

Algoritmo e tipo

`{"alg":"HS256","typ":"JWT"}`

2. Payload

Claims do usuário

`{"sub":"1","role":"admin"}`

3. Signature

Garante autenticidade

`HMAC-SHA256`

> **Hoje:** como o **Next.js** consome esse token para fazer **login, manter sessão e proteger rotas**.

🎯 Objetivos da aula

- ✅ Entender **onde armazenar** o JWT com segurança no navegador
- ✅ Criar uma **tela de login** com React Server Components + Server Actions
- ✅ Chamar a **API Node/Express com JWT** (a aula `06.JWT` do backend)
- ✅ Guardar o token em **cookie** `httpOnly`
- ✅ Proteger rotas com **Middleware (Next 15)** / **Proxy (Next 16)**
- ✅ Enviar o token em **requisições autenticadas**
- ✅ Implementar **logout** e tratamento de **token expirado**
- ✅ Aplicar as **boas práticas de segurança** da aula de JWT

❓ O grande dilema: onde guardar o JWT?

❌ localStorage / sessionStorage

- Acessível por qualquer JS da página
- Vulnerável a **XSS** (roubo do token)
- Não é enviado automaticamente

✅ Cookie httpOnly + Secure

- JavaScript **não** consegue ler o token
- Enviado automaticamente nas requisições
- Protegido contra XSS + HTTPS

⚠️ Cookie sem httpOnly

- Melhor que localStorage
- Mas ainda legível via JS
- Usar só para dados não sensíveis

Decisão da aula: o JWT será gravado em **cookie** `httpOnly` pelo Next.js — inacessível ao JavaScript do navegador.

🏗️ Arquitetura da solução



③ GET /dashboard ➜

🛡️ **Middleware / Proxy**

Verifica cookie e protege rotas

④ Authorization ➜

⚙️ **API Express + JWT**

Valida assinatura (aula 06)

📁 Estrutura do projeto (App Router)

```
next-auth-app/
├── app/
│   ├── layout.tsx
│   ├── page.tsx                    ← página pública
│   ├── login/
│   │   └── page.tsx                ← tela de login
│   ├── dashboard/
│   │   ├── layout.tsx              ← layout protegido
│   │   └── page.tsx                ← área logada
├── actions/
│   └── auth.ts                     ← Server Actions: login/logout
├── lib/
│   ├── api.ts                      ← cliente HTTP (axios/fetch)
│   └── session.ts                  ← leitura do JWT no servidor
├── middleware.ts                   ← Next 15  (Next 16: proxy.ts)
├── next.config.ts
└── package.json
```

⚙️ 1. Criando o projeto

```
npx create-next-app@latest next-auth-app
cd next-auth-app

# HTTP client (opcional — o fetch nativo também resolve)
npm install axios
```

🔑 **Sem `jose`, sem `JWT_SECRET` no Next.js!** A validação da **assinatura** do token acontece **somente na API Express** — a chave secreta **nunca sai do backend**.

Next.js 15

`App Router` · `cookies()` assíncrono · `middleware.ts`

Next.js 16

`proxy.ts` substitui `middleware.ts` · Turbopack padrão · React 19

💡 **Atenção:** desde o Next.js 15, `cookies()`, `headers()` e `params` são **assíncronos** — sempre use `await`.

🖥️ 2. Tela de login — `app/login/page.tsx`

```tsx
import { login } from '@/actions/auth';

export default function LoginPage() {
  return (
    <main style={{ maxWidth: 380, margin: '80px auto' }}>
      <h1>Entrar na plataforma</h1>

      <form action={login}>
        <label htmlFor="email">E-mail</label>
        <input id="email" name="email" type="email" required />

        <label htmlFor="senha">Senha</label>
        <input id="senha" name="senha" type="password" required />

        <button type="submit">Entrar</button>
      </form>
    </main>
  );
}
```

🎨 **Visual sugerido:** fundo claro, título em **vermelho Unipar**, botão com gradiente **#C22820 → #5C2034** (mesma identidade da aula de Backend).

🎨 Estilização com a identidade Unipar — `globals.css`

```css
:root {
  --unipar-vermelho: #C22820;   /* vermelho Unipar */
  --unipar-vinho:    #5C2034;   /* vinho Unipar  */
}

body {
  background: #f5f5f5;
  color: #272425;
  font-family: system-ui, sans-serif;
}

h1 { color: #5C2034; }

.btn-unipar {
  background: linear-gradient(135deg, #C22820, #5C2034);
  color: #fff;
  border: none;
  border-radius: 8px;
  padding: 12px 24px;
  font-size: 16px;
  cursor: pointer;
}

.btn-unipar:hover { filter: brightness(1.1); }
```

⚡ 3. Server Action de login — `actions/auth.ts`

```ts
'use server';

import { cookies } from 'next/headers';
import { redirect } from 'next/navigation';

const API = process.env.API_URL ?? 'http://localhost:3001';

export async function login(_prev: unknown, formData: FormData) {
  const email = formData.get('email');
  const senha = formData.get('senha');

  const res = await fetch(`${API}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, senha }),
  });

  if (!res.ok) {
    return { error: 'E-mail ou senha inválidos' };
  }

  const { token } = await res.json();

  const cookieStore = await cookies();          // ← await no Next 15/16
  cookieStore.set('token', token, {
    httpOnly: true,                             // ← JS não lê o token
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    path: '/',
    maxAge: 60 * 15,                            // ← 15 minutos (exp curto)
  });

  redirect('/dashboard');
}
```

🍪 4. O cookie `httpOnly` passo a passo

1️⃣ Usuário envia **email + senha** → 2️⃣ API valida e retorna **JWT assinado** → 3️⃣ Server Action grava **cookie httpOnly** → 4️⃣ Navegador segue **logado automaticamente**

⚠️ **Importante:** `cookies().set()` só funciona em **Server Actions** e **Route Handlers** — nunca em Server Components comuns (que são somente leitura).

✅ O cookie `httpOnly` **não pode ser lido por** `document.cookie` — mesmo que um ataque XSS injete JS, o token continua fora de alcance.

🛡️ 5. Protegendo rotas — Next 15 vs Next 16

Next.js 15 → `middleware.ts`

```ts
import { NextRequest, NextResponse } from 'next/server';

export function middleware(req: NextRequest) {
  const token = req.cookies.get('token')?.value;

  if (!token) {
    return NextResponse.redirect(
      new URL('/login', req.url)
    );
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*'],
};
```

Next.js 16 → `proxy.ts`

```ts
import { NextRequest, NextResponse } from 'next/server';

export function proxy(req: NextRequest) {
  const token = req.cookies.get('token')?.value;

  if (!token) {
    return NextResponse.redirect(
      new URL('/login', req.url)
    );
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*'],
};
```

> 📝 No Next.js 16 o arquivo `middleware.ts` foi renomeado para **`proxy.ts`** — a lógica continua **exatamente a mesma**.

🔍 6. Quem valida o token? Só a API Express!

🔑 o segredo (`JWT_SECRET`) . O Next.js — ele só checa se o cookie e para a API, que é a única autoridade.

```
// middleware.ts (Next 15) / proxy.ts (Next 16)
import { NextRequest, NextResponse } from 'next/server';

export function middleware(req: NextRequest) {
  const token = req.cookies.get('token')?.value;

  // ✅ Só verificamos a PRESENÇA do token (gate de entrada)
  if (!token) {
    return NextResponse.redirect(new URL('/login', req.url));
  }

  // 🔐 Assinatura e expiração? Só a API Express valida (aula 06)
  return NextResponse.next();
}
```

📡 7. Requisições autenticadas

Opção A — Server Component lendo o cookie (recomendada)

```
import { cookies } from 'next/headers';

export default async function DashboardPage() {
  const cookieStore = await cookies();
  const token = cookieStore.get('token')?.value;

  const res = await fetch(`${process.env.API_URL}/perfil`, {
    headers: { Authorization: `Bearer ${token}` },
    cache: 'no-store',
  });

  const perfil = await res.json();

  return <h1>Olá, {perfil.nome}!</h1>;
}
```

Opção B — Route Handler como proxy interno

O navegador chama `/api/perfil` do Next.js, que repassa o `Authorization` para a API — a URL do backend nunca fica exposta no cliente.

🚪 8. Logout

```
'use server';

import { cookies } from 'next/headers';
import { redirect } from 'next/navigation';

export async function logout() {
  const cookieStore = await cookies();
  cookieStore.delete('token');       // remove o cookie httpOnly
  redirect('/login');
}
```

✅ **Com cookie httpOnly**

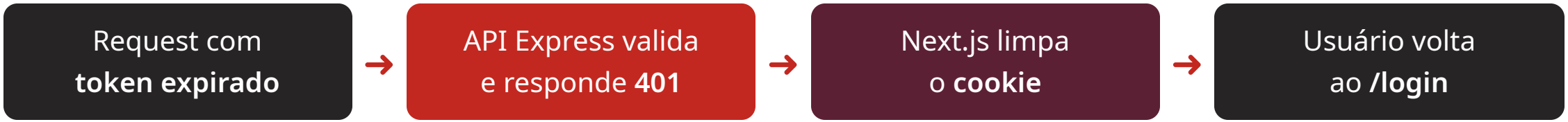
O logout acontece no **servidor**: basta apagar o cookie. Simples e seguro.

❌ **Com localStorage**

Seria preciso limpar o storage *e* confiar que o token não foi copiado — muito mais frágil.

> 💡 **Reforce a segurança:** quando a API responder `401` (token inválido ou expirado), o Next.js deve **limpar o cookie** e enviar o usuário de volta ao `/login`.

⏱️ 9. Token expirado (401)



```
// lib/api.ts — interceptor do axios
api.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401) {
      // token inválido/expirado: volta para o login
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

> ⏱️ **Sem Refresh Token por enquanto:** quando o token expirar, o usuário faz login novamente. (Futuramente podemos adicionar o fluxo de refresh — conceito da aula `06.JWT`.)

🧯 10. Erros e experiência do usuário

Use `useActionState` (React 19) para exibir erros sem recarregar a página:

```
'use client';

import { useActionState } from 'react';
import { login } from '@/actions/auth';

export default function LoginForm() {
  const [state, formAction, pending] = useActionState(login, null);

  return (
    <form action={formAction}>
      <input name="email" type="email" required />
      <input name="senha" type="password" required />

      {state?.error && (
        <p style={{ color: '#C22820' }}>{state.error}</p>
      )}

      <button disabled={pending}>
        {pending ? 'Entrando…' : 'Entrar'}
      </button>
    </form>
  );
```

🔒 11. Segurança — casando com a aula `06.JWT`

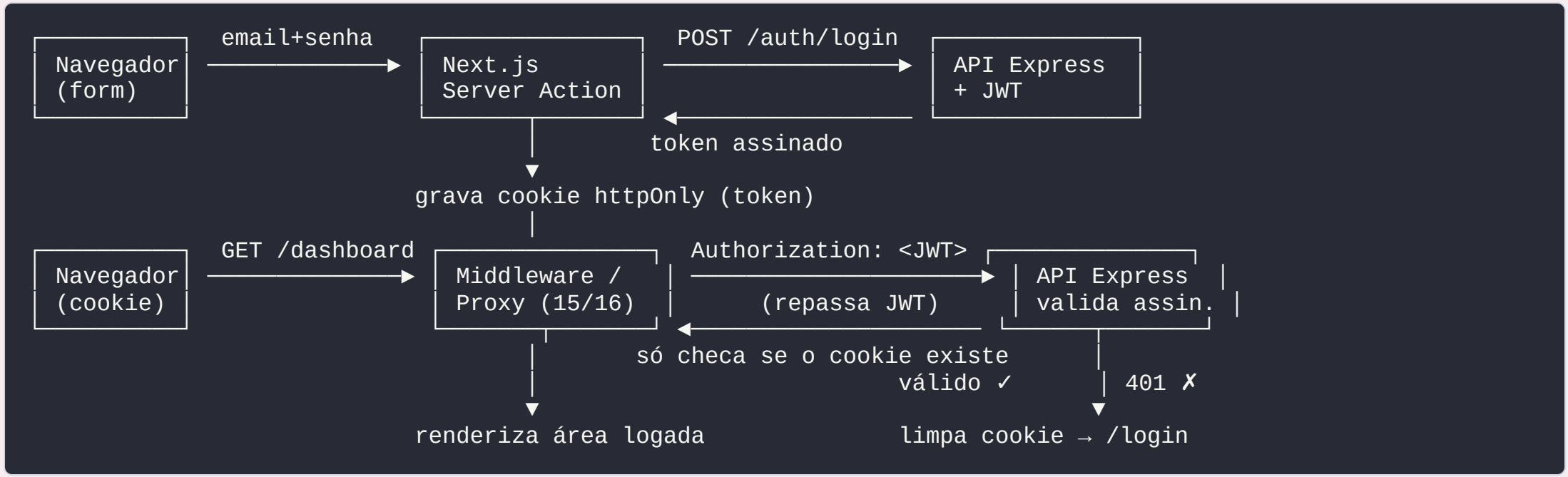
✅ Faça

- Cookie `httpOnly` + `secure` + `sameSite`
- Access token com `exp` **curto** (minutos)
- Deixar a **validação da assinatura** só na API — a chave não sai do backend
- Use **HTTPS** sempre em produção
- Mensagens de erro **genéricas** no login

❌ Evite

- Guardar JWT em `localStorage`
- Levar o `JWT_SECRET` **para o frontend** — ele vive **só no backend**
- Confiar no `alg` **do header** recebido
- Colocar **dados sensíveis no payload** (é só codificado, não criptografado!)
- Tokens de **longa duração**
- Expor detalhes internos na resposta de erro

🗺️ 12. Fluxo completo do login



Backend: aula 06.JWT

Frontend: esta aula

📦 13. Estrutura final do projeto

```
next-auth-app/
 ├─ app/
 │   ├─ login/page.tsx          ← tela de login
 │   └─ dashboard/page.tsx      ← área protegida
 ├─ actions/auth.ts             ← login() e logout()
 ├─ lib/
 │   ├─ api.ts                  ← axios/fetch com token
 │   └─ session.ts              ← leitura do cookie no servidor
 ├─ middleware.ts               ← Next 15
 │   (proxy.ts)                 ← Next 16
 ├─ .env.local                  ← só API_URL (a chave fica no backend!)
 └─ package.json
```

```
# .env.local
API_URL=http://localhost:3001

# ⚠ O JWT_SECRET fica APENAS no backend Express (aula 06.JWT)
```

🏁 Resumo e Conclusão

- 🔐 **Cookie** `httpOnly` é o lugar seguro para o JWT no Next.js
- ⚡ **Server Actions** são o lugar certo para **gravar cookies** e chamar a API
- 🔑 **Quem valida o JWT é só a API Express** — a chave secreta **nunca sai do backend**
- 🛡️ **Middleware (15) / Proxy (16)** protegem as rotas (só checam se o cookie **existe**)
- 📡 Requisições autenticadas usam o header ** `Authorization: Bearer <token>`
- 🚪 **Logout** = apagar o cookie no servidor
- ⏱️ **Token expirado (** `401` **)?** Limpa o cookie e volta ao `/login`
- 🎨 A identidade visual **Unipar** (`#C22820` e `#5C2034`) pode ser aplicada via CSS variables

> **A aula** `06.JWT` **(Backend) + esta aula (Frontend) = sistema de autenticação completo!**

💡 Desafio para praticar

- ✅ Criar a tela de login com a **identidade Unipar**
- 🛡️ Proteger a rota `/dashboard` com **middleware/proxy**
- 👤 Exibir os **dados do usuário** decodificados do JWT (claim `sub`, `nome`, `role`)
- 🚪 Criar o botão de **logout** no layout do dashboard
- 🧪 Testar: apagar o cookie manualmente e tentar acessar `/dashboard`
- 📤 Enviar o projeto no GitHub

👏 Fim da Aula

Perguntas?